

Chapter 17 作业

学号：23336128

姓名：梁力航

问题1：ACID 的含义及其在数据库系统中的重要性

说明 ACID 分别指的是什么，并说明其在数据库系统中的重要性。

回答：

- 原子性 (Atomicity)**：事务是一个不可分割的工作单位，要么全部执行，要么全部不执行。如果事务在执行过程中发生故障，已执行的操作会被回滚，保证数据库状态的一致性。重要性：确保数据库不会处于部分更新的不一致状态。
- 一致性 (Consistency)**：事务执行前后，数据库必须从一个一致状态转换到另一个一致状态。事务必须遵守所有完整性约束、业务规则和逻辑约束。重要性：保证数据的正确性和业务逻辑的有效性。
- 隔离性 (Isolation)**：并发执行的事务之间相互隔离，一个事务的执行不应影响其他事务。即使多个事务并发执行，每个事务都感觉不到其他事务的存在。重要性：防止并发事务之间的相互干扰，避免脏读、不可重复读、幻读等问题。
- 持久性 (Durability)**：一旦事务提交，其对数据库的修改就是永久性的，即使系统发生故障（如断电、崩溃），已提交的数据也不会丢失。重要性：确保数据的可靠性和可恢复性，是数据库系统可信赖的基础。

ACID 特性共同保证了数据库系统的可靠性和数据完整性，是事务处理的核心原则，对于维护数据库的正确性和可靠性至关重要。

问题2：确保持久性的存储类别

哪些存储类别可以用来确保持久性？为什么？

回答：

- 非易失性存储 (Non-volatile Storage)**：包括硬盘 (HDD)、固态硬盘 (SSD)、闪存等。这些存储介质在断电后仍能保持数据，因此可以确保持久性。数据写入这些介质后，即使系统崩溃或断电，数据也不会丢失。
- 稳定存储 (Stable Storage)**：通过冗余机制（如 RAID、镜像、备份）实现的存储系统，即使单个存储设备故障，数据仍可通过冗余副本恢复。这种存储类别提供了更高的可靠性。
- 为什么这些存储类别能确保持久性：**

- 非易失性存储的物理特性决定了数据在断电后不会丢失，这是持久性的物理基础。
- 稳定存储通过冗余和错误检测/纠正机制，提高了数据在设备故障情况下的可恢复性。
- 数据库系统通常采用写前日志（WAL）机制，先将事务的修改记录写入非易失性存储的日志文件，再写入数据文件，确保即使系统崩溃，也能通过日志恢复已提交的事务。

问题3：串行调度和可串行调度的区别

解释串行调度和可串行调度的区别。

回答：

- **串行调度 (Serial Schedule)**：事务一个接一个地执行，在任意时刻只有一个事务在执行，不存在任何并发。串行调度保证了事务的隔离性，但效率较低，无法充分利用系统资源。
- **可串行调度 (Serializable Schedule)**：多个事务并发执行，但执行结果与某个串行调度的结果相同。可串行调度在保持正确性的同时，提高了系统的并发性和吞吐量。
- **核心区别**：
 - 串行调度是物理上的串行执行，事务之间没有时间重叠；可串行调度是逻辑上的等价，允许并发执行但结果等价于串行执行。
 - 串行调度天然满足可串行性，但性能较差；可串行调度通过并发提高了性能，但需要通过冲突可串行化或视图可串行化来保证正确性。
 - 可串行调度需要满足冲突可串行化（通过交换非冲突操作）或视图可串行化（通过等价变换）的条件。

问题4：可串行化调度示例（提交顺序与串行化顺序不同）

请给出一个包含两个事务的可串行化调度的示例，使得事务提交的顺序与串行化顺序不同。

回答：

考虑两个事务 T1 和 T2：

```
T1: read(A); A := A + 10; write(A); commit;
T2: read(A); A := A * 2; write(A); commit;
初始值: A = 5
```

示例调度（T2 先提交，但串行化顺序是 T1 → T2）：

```
T1: read(A)           // A = 5
T2: read(A)           // A = 5
T1: A := A + 10       // A = 15 (局部)
T1: write(A)          // A = 15 (写入数据库)
```

```
T2: A := A * 2      // A = 10 (局部, 基于读到的 5)
T2: write(A)        // A = 10 (覆盖 T1 的写入)
T2: commit          // T2 先提交
T1: commit          // T1 后提交
最终结果: A = 10
```

等价的串行调度 $T1 \rightarrow T2$:

```
T1: read(A); A := A + 10; write(A); commit; // A = 15
T2: read(A); A := A * 2; write(A); commit; // A = 30
最终结果: A = 30
```

等价的串行调度 $T2 \rightarrow T1$:

```
T2: read(A); A := A * 2; write(A); commit; // A = 10
T1: read(A); A := A + 10; write(A); commit; // A = 20
最终结果: A = 20
```

注意: 上述示例调度实际上**不是可串行化的**, 因为最终结果 $A=10$ 既不等于 $T1 \rightarrow T2$ 的结果 (30), 也不等于 $T2 \rightarrow T1$ 的结果 (20)。

正确的可串行化调度示例:

```
T1: read(A)          // A = 5
T1: A := A + 10      // A = 15 (局部)
T1: write(A)         // A = 15
T2: read(A)          // A = 15
T2: A := A * 2       // A = 30 (局部)
T2: write(A)         // A = 30
T2: commit           // T2 先提交
T1: commit           // T1 后提交
最终结果: A = 30 (等价于串行顺序  $T1 \rightarrow T2$ )
```

在这个调度中, T2 先提交, 但执行结果等价于串行顺序 $T1 \rightarrow T2$, 因此是可串行化的。

问题5: 事务 T13 和 T14 的并发执行分析

考虑以下两个事务:

```
T13: read(A); read(B); if A = 0 then B := B + 1; write(B).
T14: read(B); read(A); if B = 0 then A := A + 1; write(A).
```

现在的一致性要求是 $A = 0 \vee B = 0$, 初始值为 $A = B = 0$ 。

回答:

a. 证明涉及这两个事务的每个串行执行都保持数据库的一致性。

串行执行 T13 → T14:

```
初始: A = 0, B = 0 (满足 A = 0 v B = 0)
T13: read(A) → A = 0
T13: read(B) → B = 0
T13: if A = 0 then B := B + 1 → B = 1
T13: write(B) → B = 1
中间状态: A = 0, B = 1 (满足 A = 0 v B = 0, 因为 A = 0)
T14: read(B) → B = 1
T14: read(A) → A = 0
T14: if B = 0 then A := A + 1 → 条件不满足, 不执行
T14: write(A) → A = 0
最终: A = 0, B = 1 (满足 A = 0 v B = 0)
```

串行执行 T14 → T13:

```
初始: A = 0, B = 0 (满足 A = 0 v B = 0)
T14: read(B) → B = 0
T14: read(A) → A = 0
T14: if B = 0 then A := A + 1 → A = 1
T14: write(A) → A = 1
中间状态: A = 1, B = 0 (满足 A = 0 v B = 0, 因为 B = 0)
T13: read(A) → A = 1
T13: read(B) → B = 0
T13: if A = 0 then B := B + 1 → 条件不满足, 不执行
T13: write(B) → B = 0
最终: A = 1, B = 0 (满足 A = 0 v B = 0)
```

因此, 每个串行执行都保持数据库的一致性。

b. 设计一个 T13 和 T14 的并发执行, 产生一个不可串行化的调度。

```
初始: A = 0, B = 0
T13: read(A)           // A = 0
T14: read(B)           // B = 0
T13: read(B)           // B = 0
T14: read(A)           // A = 0
T13: if A = 0 then B := B + 1 // 局部: B = 1
T14: if B = 0 then A := A + 1 // 局部: A = 1
T13: write(B)          // B = 1
T14: write(A)          // A = 1
最终: A = 1, B = 1 (违反一致性: A ≠ 0 且 B ≠ 0)
```

这个调度不可串行化, 因为:

- 如果等价于 T13 → T14: T13 执行后 B = 1, T14 的 if B = 0 条件不满足, A 保持为 0, 最终应为 A = 0, B = 1, 但实际是 A = 1, B = 1。

- 如果等价于 $T14 \rightarrow T13$: $T14$ 执行后 $A = 1$, $T13$ 的 $\text{if } A = 0$ 条件不满足, B 保持为 0, 最终应为 $A = 1, B = 0$, 但实际是 $A = 1, B = 1$ 。
- 因此, 这个调度不等价于任何串行执行, 且违反了数据库一致性约束。

c. 是否存在 $T13$ 和 $T14$ 的并发执行产生可串行化的调度?

答案: 是的, 存在可串行化的并发调度。

示例1: 等价于 $T13 \rightarrow T14$ 的可串行化调度

```
初始: A = 0, B = 0
T13: read(A)           // A = 0
T13: read(B)           // B = 0
T13: if A = 0 then B := B + 1 // 局部: B = 1
T13: write(B)          // B = 1
T14: read(B)           // B = 1
T14: read(A)           // A = 0
T14: if B = 0 then A := A + 1 // 条件不满足, 不执行
T14: write(A)          // A = 0
最终: A = 0, B = 1 (等价于串行顺序 T13  $\rightarrow$  T14)
```

示例2: 等价于 $T14 \rightarrow T13$ 的可串行化调度

```
初始: A = 0, B = 0
T14: read(B)           // B = 0
T14: read(A)           // A = 0
T14: if B = 0 then A := A + 1 // 局部: A = 1
T14: write(A)          // A = 1
T13: read(A)           // A = 1
T13: read(B)           // B = 0
T13: if A = 0 then B := B + 1 // 条件不满足, 不执行
T13: write(B)          // B = 0
最终: A = 1, B = 0 (等价于串行顺序 T14  $\rightarrow$  T13)
```

这些调度通过适当的操作顺序 (如先完成一个事务的写操作, 再开始另一个事务的读操作), 实现了可串行化, 同时保持了数据库的一致性。